TERRAFORM NOTES • MARCH 2026

Terraform Remote State

S3 Backend · DynamoDB Locking · IAM Policies · State Recovery

Tuesday, March 17, 2026 • OneNote Export

CONTENTS

1. Configuring the S3 Backend
2. DynamoDB Table for State Locking
3. Additional Configuration
4. IAM Policies for State Access
5. S3 Bucket Policies
6. State File Organization Strategies
7. State File Operations
8. State File Backup & Recovery
9. Common Pitfalls
10. Expert Tips from the Field
11. Key Takeaways

SECTION 01

Configuring the S3 Backend

A well-configured S3 backend is the foundation of reliable remote state management for any Terraform project. The following checklist covers all critical parameters:

- **Bucket naming** — bucket name must be lowercase alphanumeric with hyphens (no uppercase, no underscores).
- **S3 versioning** — enable versioning before the very first `terraform apply` to maintain full state file history.
- **KMS encryption** — create a customer-managed KMS key dedicated to Terraform state encryption; enables access auditing and key rotation.
- **Lifecycle policies** — configure lifecycle rules to archive old state versions (e.g. Glacier after 90 days) and to prevent accidental deletion: `lifecycle { prevent_destroy = true }`.
- **Public access block** — block all public access. This is a critical security requirement and must never be left open.

> **SECURITY CRITICAL**
>
> Blocking all public S3 access and enabling KMS encryption are non-negotiable for any production state bucket. State files can contain passwords, private keys, and API tokens.

SECTION 02

DynamoDB Table for State Locking

State locking via DynamoDB prevents concurrent `terraform apply` operations from corrupting infrastructure state. Key configuration points:

- **Billing mode** — use `PAY_PER_REQUEST` (on-demand) to avoid provisioned-capacity cost surprises; you only pay for actual locks.
- **Point-in-time recovery** — enable PITR on the lock table for an additional safety net.
- **Prevent accidental deletion** — apply `lifecycle { prevent_destroy = true }` to the DynamoDB resource just as you do for the S3 bucket.
- **Monitoring stuck locks** — set a CloudWatch alarm to fire when a lock item exists for more than 1 hour; this indicates a crashed Terraform process still holding the lock.

SECTION 03

Additional Configuration

Migrating from Local to Remote State

After configuring the backend block, run `terraform init`. Terraform will detect the backend change and prompt you to migrate existing local state to the remote S3 location. Always take a manual backup of the local `terraform.tfstate` file before migrating.

Automatic Lock Timeout (Best Practice)

Configure an automatic lock timeout so that a stale lock does not block the team indefinitely. Combine this with the CloudWatch alarm described above to ensure that stuck locks are detected and resolved quickly.

State File Backup

Even with S3 versioning enabled, consider a secondary backup strategy — for example a Lambda function triggered on state file changes that copies the latest version to a separate bucket or cross-region location, with SNS alerts on failure.

SECTION 04

IAM Policies for State Access

Scope IAM permissions tightly. The principle of least privilege is especially important for state files because they contain sensitive infrastructure data.

Read-Only Access (Developers Reviewing Infrastructure)

- `s3:GetObject` on the state bucket
- `s3:ListBucket` on the state bucket
- `dynamodb:GetItem` on the lock table (to inspect lock status)

Full Access (CI/CD & Infrastructure Team)

- `s3:GetObject`, `s3:PutObject`, `s3:DeleteObject` on the state bucket
- `s3:ListBucket` on the state bucket
- `dynamodb:GetItem`, `dynamodb:PutItem`, `dynamodb:DeleteItem` on the lock table
- `kms:GenerateDataKey`, `kms:Decrypt` on the KMS key

> **ADVANCED HARDENING**
>
> Enhance IAM policies with conditions: restrict state access to specific IP ranges, require MFA for destructive operations, or impose time-of-day restrictions for production state access.

SECTION 05

S3 Bucket Policies

Beyond IAM user/role policies, apply a bucket policy that:

- **Enforces encryption in transit** — deny any request that does not use `aws:SecureTransport: true` (i.e. non-HTTPS).

- **Restricts principal access** — allow only the specific IAM roles used by Terraform and CI/CD.
- **Enables MFA Delete on production** — requires an MFA token to permanently delete state file versions, preventing accidents and unauthorized deletions.

SECTION 06

State File Organization Strategies

Strategy 1 — By Environment

Each environment gets its own backend configuration pointing to a dedicated S3 key prefix or even a separate bucket:

- `environments/dev/backend.tf`
- `environments/staging/backend.tf`
- `environments/prod/backend.tf`

This approach minimizes blast radius: a bad `apply` in dev cannot reach prod state.

Strategy 2 — By Team / Service

Separate state files per team or microservice. Network infrastructure rarely changes; application services change frequently. Keeping them in separate state files means network locks never block application deployments.

> **WORKSPACE NOTE**
>
> Terraform workspaces are powerful for same-infrastructure/different-configuration scenarios within a single AWS account. They should *not* be used as a substitute for completely separate accounts or fundamentally different architectures. Use `workspace_key_prefix` to give your workspace states a cleaner S3 path (e.g. `"workspaces"` instead of the default `"env:"`).

SECTION 07

State File Operations

- **List all resources in state** — `terraform state list`

- **Show detailed resource information** — `terraform state show <resource>`
- **Pull state locally** — `terraform state pull > local.tfstate` (always do this before risky bulk operations)
- **Push state remotely** — `terraform state push local.tfstate` (requires verified IAM permissions)
- **Move resource in state** — `terraform state mv <old> <new>`
- **Remove resource from state** — `terraform state rm <resource>` (does not destroy the real resource)

> **BEST PRACTICE**
>
> Always run `terraform state pull > backup.tfstate` before any bulk `import`, `mv`, or `rm` operation. Recovery from a local copy takes seconds; manual reconstruction takes hours.

SECTION 08

State File Backup & Recovery

Automated Backup

- Create a Lambda function triggered on S3 state-file change events.
- Lambda copies the new version to a secondary bucket (optionally cross-region).
- Wire an SNS topic for failure alerts.

Scenario 1 — Corrupted State File

Solution: Restore from S3 versioning.

1. List versions: `aws s3api list-object-versions --bucket <bucket> --prefix <key>`
2. Download a specific version: `aws s3api get-object --version-id <id> ...`
3. Verify recovered state: `terraform state list`
4. Push if verified: `terraform state push recovered.tfstate`

Scenario 2 — Accidentally Deleted State

1. State file deleted from S3 — locate the delete marker via `list-object-versions`.
2. Remove the delete marker to restore the latest version.

3. Verify with `terraform state list`.

Scenario 3 — State Drift (Resources Changed Outside Terraform)

Detect drift by running `terraform plan`; it will show resources that differ from state.

- **Option 1** — Update state to match reality: `terraform import` or `terraform refresh`.
- **Option 2** — Revert AWS changes to match Terraform: apply the original configuration.
- **Option 3** — Update Terraform config to match AWS: edit HCL and re-plan.

SECTION 09

Common Pitfalls

#	Pitfall	Why It Matters
1	Not Enabling Versioning	No recovery path if state is corrupted before versioning is turned on.
2	Forgetting State Locking	Concurrent applies can corrupt state, leading to resource conflicts or orphaned infrastructure.
3	Storing Sensitive Data Unencrypted	State files contain secrets; unencrypted state in S3 is a compliance and security risk.
4	Single State File for Everything	Blast radius of any failed apply is your entire infrastructure. Splits reduce risk.
5	Not Backing Up State Outside S3	S3 bucket policy misconfiguration or accidental bucket deletion can wipe all versions.
6	Mixing Workspaces with Separate Accounts	Workspaces share the same backend bucket; cross-account state introduces permission complexity.
7	Not Testing State Recovery	Untested procedures fail under pressure. Quarterly recovery drills are essential.
8	Hardcoding Backend Configuration	Prevents reuse across environments. Use partial backend config with `-backend-config`.

9	Ignoring State File Size	States over 10 MB slow all operations; split by service, team, or lifecycle.
10	Not Monitoring State Access	Unauthorized or unexpected state access goes undetected without CloudWatch/CloudTrail alarms.

SECTION 10

Expert Tips from the Field

1. **Enable S3 versioning BEFORE first `terraform apply`**

   You cannot retroactively version previous states. A first apply without versioning means no recovery if it fails.

2. **Use `workspace_key_prefix` to organize workspace states**

   The default `env:/` path is confusing. Set `workspace_key_prefix = "workspaces"` for a cleaner S3 structure.

3. **DynamoDB on-demand pricing prevents lock cost surprises**

   `PAY_PER_REQUEST` billing mode means you only pay for actual locks, not provisioned capacity sitting idle.

4. **Test state push/pull permissions before production deployment**

   Run `terraform state pull > test.tfstate` and `terraform state push test.tfstate` to verify IAM permissions work end-to-end.

5. **Separate network and application state files**

   Network changes rarely; apps change frequently. Separate states prevent network locks during app deployments.

6. **Use `terraform_remote_state` data source, not manual copying**

   Cross-state references via outputs keep dependencies explicit and prevent drift between modules.

7 **Set `lifecycle prevent_destroy` on the state bucket**

Accidental `terraform destroy` of state infrastructure means losing all state files. Always protect with lifecycle.

8 **Enable MFA Delete on production state bucket**

Requires an MFA token to delete state file versions. Prevents accidents and unauthorized deletions.

9 **Backend configuration belongs in `backend.tf`, not `main.tf`**

Standard file structure makes backend changes immediately obvious to the entire team.

10 **Use partial backend config for multi-environment**

Define backend type in code, pass bucket/key via `-backend-config`. Eliminates copy-paste errors across environment directories.

11 **State file size over 10 MB indicates need for splitting**

Large states slow all operations. Split by service, team, or lifecycle for better performance.

12 **Always run `terraform state pull` before risky operations**

Local backup before bulk imports, moves, or refactoring. Recovery takes seconds, not hours.

13 **Monitor DynamoDB lock table for stuck locks**

CloudWatch alarm when an item exists for more than 1 hour indicates a crashed Terraform process still holding the lock.

14 **Use S3 Glacier for old state versions after 90 days**

Lifecycle policy archives old versions, reducing storage costs 90%+ while maintaining recovery ability.

15 **Document state recovery procedures in a runbook**

A 3 AM production incident is not the time to learn S3 versioning. Test and document recovery steps quarterly.

16 **Encrypt state with a customer-managed KMS key**

AWS-managed encryption is good; a CMK is better. It enables access auditing and key rotation policies.

17 **Use separate state buckets per AWS account**

Dev account state lives in the dev account bucket. Prevents cross-account permission complexity.

18

Set S3 Intelligent Tiering on the state bucket

Automatically moves infrequently accessed states to cheaper storage tiers without manual lifecycle rules.

19

Never manually edit the state file in S3

Always use `terraform state` commands. Manual S3 edits bypass serial-number increments and can corrupt state.

20

Create state access IAM policies with conditions

Restrict state access to specific IP ranges, require MFA, or apply time-of-day restrictions for production state.

Key Takeaways

- Remote state with S3 backend and DynamoDB locking is **essential for team collaboration**, preventing simultaneous modifications that could corrupt infrastructure state.
- S3 versioning is your **safety net for state file disasters**, enabling recovery from corruption or accidental deletion in minutes rather than hours of manual reconstruction.
- State file encryption with AWS KMS is **non-negotiable for production environments**, as state files contain sensitive data including passwords, private keys, and API tokens.
- Organizing state files by environment, team, or lifecycle **reduces blast radius** of changes and enables granular access control through IAM policies.
- State locking with DynamoDB prevents concurrent modifications, but **requires monitoring for stuck locks** that can block deployments after process crashes.
- Workspaces are powerful for same-infrastructure/different-configuration scenarios within a single AWS account, but **should not be used for completely different accounts or architectures**.
- Regular state recovery drills verify your backup strategy works under pressure, ensuring **3 AM production incidents don't become multi-hour disasters**.